

02_Alzheimer_Cognitive_fonctions

August 28, 2025

```
[1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
from sklearn.model_selection import train_test_split, GridSearchCV, StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, classification_report, confusion_matrix)

warnings.filterwarnings('ignore')
```

```
[2]: # FONCTIONS DE CHARGEMENT ET PRÉ-TRAITEMENT DES DONNÉES

def load_alzheimer_data(filepath):
    """
    Charger le jeu de données Alzheimer depuis le fichier CSV
    """
    try:
        df = pd.read_csv(filepath)
        print(f"Jeu de données chargé avec succès. Forme : {df.shape}")
        return df
    except FileNotFoundError:
        print(f"Erreur : fichier {filepath} introuvable.")
        return None
    except Exception as e:
        print(f"Erreur lors du chargement des données : {str(e)}")
        return None

def preprocess_data(df, columns_to_drop=['PatientID', 'DoctorInCharge']):
    """
    Pré-traiter le jeu de données en supprimant les colonnes inutiles
    """
```

```

"""
df_clean = df.drop(columns_to_drop, axis=1)
print(f"Forme du jeu de données après pré-traitement : {df_clean.shape}")
print(f"Colonnes supprimées : {columns_to_drop}")
return df_clean

def check_data_quality(df):
    """
    Vérifier la qualité des données, valeurs manquantes et statistiques de base
    """
    quality_report = {
        'shape': df.shape,
        'missing_values': df.isnull().sum(),
        'data_types': df.dtypes,
        'duplicates': df.duplicated().sum()
    }

    print("Rapport de qualité des données :")
    print(f"Forme : {quality_report['shape']}")
    print(f"Valeurs manquantes : \n{quality_report['missing_values']}")
    print(f"Doublons : {quality_report['duplicates']}")

    return quality_report

```

[3]: # FONCTIONS DE DÉTECTION DES VALEURS ABERRANTES

```

def detect_outliers_iqr(dataframe, target_column='Diagnosis'):
    """
    Détecter les valeurs aberrantes avec la méthode de l'écart interquartile,
    ↪ (IQR)
    """
    outlier_indices = []
    feature_columns = [col for col in dataframe.columns if col != target_column]

    for col in feature_columns:
        Q1 = dataframe[col].quantile(0.25)
        Q3 = dataframe[col].quantile(0.75)
        IQR = Q3 - Q1
        outlier_step = 1.5 * IQR

        outliers = dataframe[
            (dataframe[col] < Q1 - outlier_step) |
            (dataframe[col] > Q3 + outlier_step)
        ].index

        outlier_indices.extend(outliers)

```

```

outlier_indices = list(set(outlier_indices))
print(f"Nombre de valeurs aberrantes détectées : {len(outlier_indices)}")

return outlier_indices

def remove_outliers(dataframe, outlier_indices):
    """
    Supprimer les valeurs aberrantes du dataframe
    """
    df_clean = dataframe.drop(outlier_indices)
    print(f"{len(outlier_indices)} valeurs aberrantes supprimées")
    print(f"Nouvelle forme du jeu de données : {df_clean.shape}")

    return df_clean

```

[4]: # FONCTIONS DE VISUALISATION

```

def plot_feature_distributions(dataframe, target_column='Diagnosis',
    figsize=(40, 45)):
    """
    Plot la distribution avec hue.
    """
    feature_cols = [col for col in dataframe.columns if col != target_column]
    n_features = len(feature_cols)
    n_cols = 3
    n_rows = (n_features // n_cols) + (1 if n_features % n_cols != 0 else 0)

    palette = sns.color_palette("Set2") # même palette que l'histogramme
    value_to_color = dict(zip(sorted(dataframe[target_column].unique()),
    palette))

    fig = plt.figure(figsize=figsize)

    for idx, col in enumerate(feature_cols, start=1):
        ax = plt.subplot(n_rows, n_cols, idx)

        # Histogramme
        sns.histplot(
            data=dataframe,
            x=col,
            hue=target_column,
            kde=True,
            palette=value_to_color,
            alpha=0.5,
            ax=ax,
            multiple='stack'
        )

```

```

# Lignes de moyenne et médiane
for target_value, color in value_to_color.items():
    subset = dataframe[dataframe[target_column] == target_value]
    mean_val = np.mean(subset[col])
    median_val = np.median(subset[col])

    ax.axvline(
        mean_val,
        color=color,
        linestyle='--',
        linewidth=2,
        label=f"{target_column}={target_value} (moyenne)"
    )
    ax.axvline(
        median_val,
        color=color,
        linestyle='-',
        linewidth=2,
        label=f"{target_column}={target_value} (médiane)"
    )

ax.set_title(f'Distribution de {col}')
# Fusionner les légendes
handles, labels = ax.get_legend_handles_labels()
ax.legend(handles, labels, title=f"{target_column}")

plt.tight_layout()
plt.show()

def plot_correlation_heatmap(dataframe, figsize=(20, 15), annot=False):
    """
    Plot la carte de corrélation
    """
    corr_matrix = dataframe.corr()

    plt.figure(figsize=figsize)
    sns.heatmap(corr_matrix, annot=annot, cmap='coolwarm')
    plt.title("Matrice de corrélation")
    plt.show()

    return corr_matrix

def plot_feature_importance(feature_scores, title="Importance des_
↪caractéristiques",
                            figsize=(10, 15)):
    """

```

```

Plot les scores d'importance des caractéristiques
"""

plt.figure(figsize=figsize)
sns.barplot(x=feature_scores.values, y=feature_scores.index)
plt.title(title)
plt.xlabel("Importance")
plt.ylabel("Caractéristiques")
plt.show()

```

[5]: *# FONCTIONS DE DIVISION ET MISE À L'ÉCHELLE DES DONNÉES*

```

def split_and_scale_data(X, y, test_size=0.2, random_state=1):
    """
    Diviser les données en ensembles d'entraînement/test et appliquer la
    ↪standardisation
    """

    # Diviser les données
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=test_size, random_state=random_state, stratify=y
    )

    # Standardiser les caractéristiques
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    # Convertir en DataFrames
    X_train_scaled = pd.DataFrame(X_train_scaled, columns=X_train.columns)
    X_test_scaled = pd.DataFrame(X_test_scaled, columns=X_test.columns)

    print(f"Forme de l'ensemble d'entraînement : {X_train_scaled.shape}")
    print(f"Forme de l'ensemble de test : {X_test_scaled.shape}")

    return X_train_scaled, X_test_scaled, y_train, y_test, scaler

def analyze_target_distribution(y):
    """
    Analyser la distribution de la variable cible
    """

    unique_classes, class_counts = np.unique(y, return_counts=True)

    print("Distribution des classes :")
    for cls, count in zip(unique_classes, class_counts):
        percentage = (count / len(y)) * 100
        print(f"Classe {cls} : {count} échantillons ({percentage:.2f}%)")

    return dict(zip(unique_classes, class_counts))

```

[6]: # FONCTIONS DE SÉLECTION DES CARACTÉRISTIQUES

```
def select_features_by_correlation(X_train_scaled, y_train, threshold=0.1):  
    """  
    Sélectionner les caractéristiques selon leur corrélation avec la cible  
    """  
    # Calculer la matrice de corrélation  
    corr_matrix = X_train_scaled.join(y_train).corr()  
  
    # Identifier les caractéristiques corrélées avec la cible  
    target_corr = abs(corr_matrix['Diagnosis'])  
    correlated_features = target_corr[target_corr > threshold].index.tolist()  
  
    if 'Diagnosis' in correlated_features:  
        correlated_features.remove('Diagnosis')  
  
    print(f"Caractéristiques sélectionnées par corrélation ( $|r| > \{threshold\}$ ) :  
↳ {len(correlated_features)}")  
    print(correlated_features)  
  
    return correlated_features  
  
def select_features_by_rf_importance(X_train_scaled, y_train, threshold=0.02,  
                                     n_estimators=100, random_state=1):  
    """  
    Sélectionner les caractéristiques via l'importance Random Forest  
    """  
    # Entraîner Random Forest  
    rf = RandomForestClassifier(n_estimators=n_estimators,   
↳ random_state=random_state)  
    rf.fit(X_train_scaled, y_train)  
  
    # Obtenir l'importance des caractéristiques  
    feature_importances = pd.Series(  
        rf.feature_importances_,  
        index=X_train_scaled.columns  
    ).sort_values(ascending=False)  
  
    # Sélectionner les caractéristiques au-dessus du seuil  
    selected_features = feature_importances[feature_importances > threshold].  
↳ index.tolist()  
  
    print(f"Caractéristiques sélectionnées par importance RF ( $> \{threshold\}$ ) :  
↳ {len(selected_features)}")  
    print(selected_features)  
  
    # Plot de l'importance
```

```

    plot_feature_importance(feature_importances, "Importance des_
↳caractéristiques (Random Forest)")

    return selected_features, feature_importances

def select_features_by_anova(X_train_scaled, y_train, threshold=10):
    """
    Sélectionner les caractéristiques via le test F ANOVA
    """
    # Test F ANOVA
    selector = SelectKBest(score_func=f_classif, k='all')
    selector.fit(X_train_scaled, y_train)

    # Obtenir les scores
    feature_scores = pd.Series(
        selector.scores_,
        index=X_train_scaled.columns
    ).sort_values(ascending=False)

    # Sélectionner les caractéristiques au-dessus du seuil
    selected_features = feature_scores[feature_scores > threshold].index.
↳tolist()

    print(f"Caractéristiques sélectionnées par ANOVA (F-score > {threshold}) :_
↳{len(selected_features)}")
    print(selected_features)

    # Plot des scores
    plot_feature_importance(feature_scores, "Scores des caractéristiques (Test_
↳F ANOVA)")

    return selected_features, feature_scores

def combine_selected_features(*feature_lists):
    """
    Combiner les résultats de sélection de caractéristiques
    """
    combined_features = []
    for feature_list in feature_lists:
        combined_features.extend(feature_list)

    # Supprimer les doublons en préservant l'ordre
    unique_features = list(dict.fromkeys(combined_features))

    print(f"Total de caractéristiques uniques après combinaison :_
↳{len(unique_features)}")
    print(unique_features)

```

```
return unique_features
```

[7]: # FONCTIONS D'ENTRAÎNEMENT ET ÉVALUATION DES MODÈLES

```
def get_default_models():
    """
    Obtenir le dictionnaire des modèles par défaut pour comparaison
    """
    models = {
        'Régression Logistique': LogisticRegression(max_iter=10000),
        'Forêt Aléatoire': RandomForestClassifier(),
        'SVM': SVC(probability=True)
    }
    return models

def evaluate_model(model, X_train, y_train, X_test, y_test,
    model_name="Modèle"):
    """
    Évaluer un modèle de machine learning
    """
    # Entraîner le modèle
    model.fit(X_train, y_train)

    # Prédiction
    y_pred = model.predict(X_test)

    # Calculer les métriques
    results = {
        'model': model,
        'accuracy': accuracy_score(y_test, y_pred),
        'precision': precision_score(y_test, y_pred),
        'recall': recall_score(y_test, y_pred),
        'f1': f1_score(y_test, y_pred),
        'classification_report': classification_report(y_test, y_pred),
        'confusion_matrix': confusion_matrix(y_test, y_pred)
    }

    # Afficher les résultats
    print(f"\nRésultats pour {model_name} :")
    print(f"Accuracy : {results['accuracy']:.4f}")
    print(f"Précision : {results['precision']:.4f}")
    print(f"Rappel : {results['recall']:.4f}")
    print(f"Score F1 : {results['f1']:.4f}")
    print(f"\nRapport de classification :")
    print(results['classification_report'])
    print(f"\nMatrice de confusion :")
```

```

print(results['confusion_matrix'])

return results

def evaluate_multiple_models(models, X_train, y_train, X_test, y_test):
    """
    Évaluer plusieurs modèles et retourner les résultats
    """
    results = {}

    for model_name, model in models.items():
        print(f"\nEntraînement et évaluation de {model_name}")
        results[model_name] = evaluate_model(
            model, X_train, y_train, X_test, y_test, model_name
        )

    return results

```

[8]: # FONCTIONS D'OPTIMISATION DES HYPERPARAMÈTRES

```

def get_default_param_grids():
    """
    Obtenir les grilles de paramètres par défaut pour l'optimisation
    """
    param_grids = {
        'Régression Logistique': {
            'C': [0.001, 0.01, 0.1, 1, 10, 100],
            'penalty': ['l2']
        },
        'Forêt Aléatoire': {
            'n_estimators': [50, 100, 200],
            'max_depth': [None, 10, 20, 30],
            'min_samples_split': [2, 5, 10],
            'min_samples_leaf': [1, 2, 4]
        },
        'SVM': {
            'C': [0.1, 1, 10, 100],
            'kernel': ['linear', 'rbf'],
            'gamma': ['scale', 'auto', 0.1, 1, 10]
        }
    }
    return param_grids

def optimize_hyperparameters(model, param_grid, X_train, y_train,
                             scoring='f1', cv=5, n_jobs=-1, random_state=1):
    """
    Optimiser les hyperparamètres avec GridSearchCV
    """

```

```

"""
# Validation croisée stratifiée
cv_splitter = StratifiedKFold(n_splits=cv, shuffle=True,
↪random_state=random_state)

# GridSearchCV
grid_search = GridSearchCV(
    estimator=model,
    param_grid=param_grid,
    scoring=scoring,
    cv=cv_splitter,
    n_jobs=n_jobs,
    verbose=1
)

# Ajuster sur les données d'entraînement
grid_search.fit(X_train, y_train)

# Extraire les résultats
best_model = grid_search.best_estimator_
best_params = grid_search.best_params_
best_score = grid_search.best_score_

print(f"Meilleurs paramètres : {best_params}")
print(f"Meilleur score {scoring} : {best_score:.4f}")

return best_model, best_params, best_score

def optimize_all_models(models, param_grids, X_train, y_train,
                        X_test, y_test, scoring='f1'):
    """
Optimiser les hyperparamètres pour tous les modèles
"""
    optimized_results = {}

    for model_name, model in models.items():
        print(f"\n{'='*60}")
        print(f"Optimisation de {model_name}")
        print(f"{'='*60}")

        # Optimiser les hyperparamètres
        best_model, best_params, best_score = optimize_hyperparameters(
            model, param_grids[model_name], X_train, y_train, scoring=scoring
        )

        # Évaluer le modèle optimisé
        print(f"\nÉvaluation de {model_name} optimisé")

```

```

    results = evaluate_model(
        best_model, X_train, y_train, X_test, y_test,
        f"{model_name} (optimisé)"
    )

    # Ajouter les infos d'optimisation aux résultats
    results['best_params'] = best_params
    results['best_cv_score'] = best_score

    optimized_results[f"{model_name} (optimisé)"] = results

    return optimized_results

```

[9]: *# FONCTIONS DE COMPARAISON DES RÉSULTATS*

```

def create_comparison_dataframe(results_dict):
    """
    Créer un DataFrame de comparaison depuis le dictionnaire de résultats
    """
    comparison_data = []

    for model_name, results in results_dict.items():
        comparison_data.append({
            'Modèle': model_name,
            'Accuracy': results['accuracy'],
            'Précision': results['precision'],
            'Rappel': results['recall'],
            'Score F1': results['f1']
        })

    return pd.DataFrame(comparison_data)

def find_best_model(comparison_df, metric='Score F1'):
    """
    Trouver le meilleur modèle selon la métrique spécifiée
    """
    best_idx = comparison_df[metric].idxmax()
    best_model_name = comparison_df.loc[best_idx, 'Modèle']
    best_score = comparison_df.loc[best_idx, metric]

    print(f"\nMeilleur modèle selon {metric} :")
    print(f"Modèle : {best_model_name}")
    print(f"{metric} : {best_score:.4f}")

    return best_model_name, best_score

def plot_model_comparison(comparison_df, figsize=(12, 8)):

```

```

"""
Plot la comparaison des performances des modèles
"""
# Transformer le dataframe pour le graphique
metrics_df = comparison_df.melt(
    id_vars='Modèle',
    value_vars=['Accuracy', 'Précision', 'Rappel', 'Score F1'],
    var_name='Métrique',
    value_name='Score'
)

plt.figure(figsize=figsize)
sns.barplot(data=metrics_df, x='Modèle', y='Score', hue='Métrique')
plt.title('Comparaison des performances des modèles')
plt.ylabel('Score')
plt.xlabel('Modèle')
plt.xticks(rotation=45)
plt.legend(title='Métriques')
plt.tight_layout()
plt.show()

```

```

[10]: def save_model_results(results_dict, filepath):
    """
    Sauvegarder les résultats des modèles dans un fichier
    """
    import pickle

    # Supprimer les objets sklearn pour la sérialisation
    serializable_results = {}
    for model_name, results in results_dict.items():
        serializable_results[model_name] = {
            key: value for key, value in results.items()
            if key not in ['model']
        }

    with open(filepath, 'wb') as f:
        pickle.dump(serializable_results, f)

    print(f"Résultats sauvegardés dans {filepath}")

```

```

[11]: def print_summary_report(comparison_df, best_model_name):
    """
    Afficher un rapport récapitulatif complet
    """
    print("\n" + "="*80)
    print("ANALYSE COGNITIVE ALZHEIMER - RAPPORT RÉCAPITULATIF")
    print("="*80)

```

```

print(f"\nNombre de modèles évalués : {len(comparison_df)}")
print(f"\nMeilleur modèle : {best_model_name}")

print(f"\nRésumé des performances des modèles :")
print(comparison_df.round(4))

print(f"\nStatistiques de performance :")
print(f"Score F1 moyen : {comparison_df['Score F1'].mean():.4f}")
print(f"Meilleur score F1 : {comparison_df['Score F1'].max():.4f}")
print(f"Pire score F1 : {comparison_df['Score F1'].min():.4f}")
print(f"Écart-type : {comparison_df['Score F1'].std():.4f}")

print("\n" + "="*80)

```